

우당탕 타워

AI와 함께 개발 루프를 설계한 기록

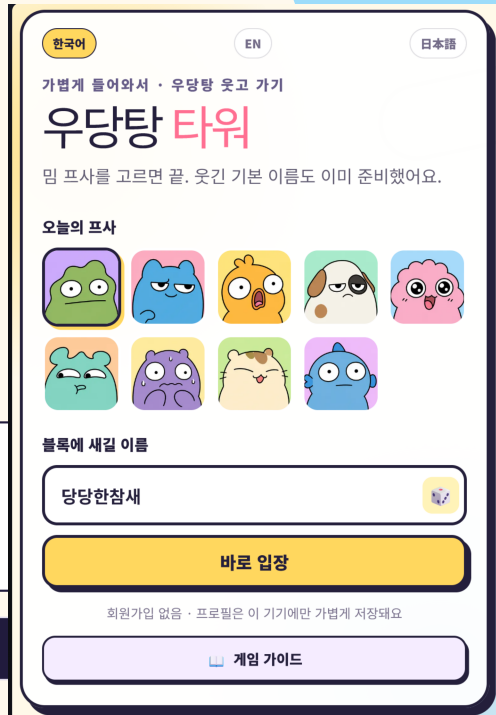
바이트코딩을 결과물의 변명이 아니라, 게임 프로그래머의 판단을 더 빠르게 구현·검증·수정하기 위한 개발 방식으로 사용한 v0.1 사례 보고서.

ROLE

박찬욱 · Game Client Programmer

게임 기획 · 클라이언트/물리 · AI 개발 디렉팅 · 출시 판단

VERSION	PLATFORM	LOCALIZATION
0.1	Web · Mobile	KO · EN · JP



이 문서가 보여주는 것

AI가 코드를 얼마나 많이 작성했는지가 아니라, 개발자가 문제를 정의하고 감각을 규칙으로 바꾸며, 증거를 통해 구현을 거절·수정하고, 테스트와 배포까지 하나의 루프로 운영한 방식을 기록한다.

AI를 도구로 쓰고, 개발 루프는 내가 설계했다

구현 속도는 AI가 높였지만 무엇을 만들지, 어떤 결과를 통과로 볼지, 언제 롤백하고 출시할지는 게임 프로그래머가 결정했다.

이 프로젝트에서 AI는 **대신 판단하는 주체**가 아니라 빠른 시제품 제작, 코드 수정, 리소스 변형, 테스트 보강을 담당하는 실행 도구였다. 나는 플레이 중 느낀 불편을 그대로 넘기지 않고, 왜 불공정하게 느껴졌는지 규칙과 상태로 분해했다. 예를 들어 '난이도가 너무 어렵다'는 피드백은 이동 해상도, 낙하 속도, 첫 충돌 충격, 안정 보조의 해제 조건, 젠가 마찰처럼 서로 독립적으로 검증 가능한 항목으로 바뀌었다.

대화는 기획서의 대체물이 아니라 **살아 있는 요구사항 정제 과정**이었다. 스크린샷에는 잘못된 장면과 정상 기대값을 함께 적고, 플레이 영상에는 도달 높이와 체감 목표를 붙였다. AI가 제안한 결과가 의도와 다르면 '별로'라고 끝내지 않고 돌출된 얼굴, Z-fighting, 잘못된 충돌 기준, 입력 충돌처럼 거절 사유를 기술 언어로 남겼다. 덕분에 다음 수정은 취향 맞추기가 아니라 조건 충족 작업이 됐다.

또한 빠른 반복이 구조적 부채로 바뀌지 않도록 규칙 모듈, 테스트, Git 커밋, HANDOFF, 실제 공개 번들 확인을 함께 관리했다. 현재 v0.1은 싱글 플레이 기반의 로컬 빌드이지만, 다음 버전의 프라이빗 룸과 최대 20인 공개 멀티를 고려해 판정 책임과 상태 경계를 분리하고 있다.

오해하기 쉬운 한 문장을 이렇게 고쳐 쓴다

“AI가 만들고 내가 고른 것이 아니다”가 아니라, “AI에게 무엇을 시킬지보다 무엇을 통과해야 하는지 먼저 정했다.”

개발자가 직접 책임진 것	AI가 가속한 것
재미의 기준, 게임 규칙, 물리 허용 범위, 우선순위, 최종 UX, 출시 여부	구현 초안, 반복 수정, 리소스 시안, 코드 탐색, 테스트 추가, 문서화와 배포 작업

웃음을 만드는 3D 물리 스택 게임

높이를 올리는 행위만이 아니라, 잘 쌓인 탑과 예상 밖의 붕괴를 모두 소셜 사건으로 만드는 것이 핵심이다.



v0.1 게임 화면. 블록 배치, 실시간 점수, 채팅, 해설, 고도 UI와 모바일 대응 조작을 한 화면에 통합했다.

가벼운 진입

게스트 우선, 밌 프로필, 자동 생성 닉네임, 로컬 프로필 저장으로 선택 피로와 로딩을 줄였다.

관전도 플레이

차례가 아니어도 채팅, 실시간 점수, 해설, 젠가 번호를 통해 계속 상황에 참여한다.

물리가 서사

성공 배치, 간신히 버티는 탑, 단일 이탈과 구조 붕괴가 점수와 연출의 원인이 된다.

현재 공개판은 **싱글 플레이로 물리와 조작을 검증하는 v0.1**이다. 블록은 차례 시작 시 공개되고, 60초 동안 위치와 Y축 회전, 가로/세로 자세를 조정해 놓는다. 중앙 정렬과 안정성에 따라 점수를 얻고, 경기장을 이탈한 블록은 소유자 점수에서 차감된다. 일정 조건이 되면 젠가 이벤트가 발생하며, 특수 문어 블록은 아래 블록을 붙잡아 탑의 상황을 바꾼다.

초기 3개 블록과 현재 탑의 수평 지지 범위 안에서만 배치가 인정된다. 이는 맨바닥에 새 탑을 만드는 우회 플레이를 막고, '모두가 한 탑을 이어 만든다'는 규칙을 물리 판정과 일치시키기 위한 결정이다. 공개 멀티 이전에 싱글에서 이 규칙을 충분히 검증하는 이유도 서버 동기화 전에 재미의 기준점을 고정하기 위해서다.

현재 공개 빌드

udangtang-tower.pcw0611.workers.dev · TypeScript · Three.js · cannon-es · WebGL · Cloudflare

대화를 작업 지시가 아니라 검증 가능한 루프로 만든다

한 번에 완성품을 요구하지 않는다. 작은 변경을 만들고, 플레이 증거로 틀린 점을 좁히며, 통과 기준을 코드와 테스트에 남긴다.

단계	내가 제공한 것	AI가 수행한 것	다음 단계로 가는 조건
1. 문제 포착	플레이 감각, 스크린샷, 영상, 현재 URL	관련 코드와 상태 흐름 탐색	현상과 기대값이 같은 문장에 있음
2. 규칙화	재미의 의도, 금지 조건, 예외	상태·수치·책임 모듈로 번역	구현 전 판정 기준을 설명 가능
3. 작은 구현	이번 슬라이스의 범위	최소 변경, 필요한 리소스/테스트 추가	기존 동작을 불필요하게 건드리지 않음
4. 플레이 검증	직접 조작, 캡처, 체감 비교	원인 분석, 수정 또는 롤백	같은 장면에서 기대 행동 재현
5. 출시	버전과 우선순위 판단	빌드·테스트·커밋·배포·공개 확인	소스와 실제 공개 번들이 일치

컨텍스트는 최대한 구체적으로 묶어 전달한다. 한 장의 캡처만 보내기보다 현재 화면, 직전 의도, 정상 기대값, 롤백 여부를 함께 준다. '카메라가 어색하다'는 말도 '블록을 따라 내려가는 방식은 롤백하고 기존 꼭대기 추적 방식으로 복귀'처럼 비교 기준을 포함한다. AI가 대화의 분위기만 따라 임의로 기능을 넓히지 못하게 하는 장치다.

역할을 나눠 문맥과 위험을 관리한다. 리소스 생성, 메인 구현·통합, 범위가 좁은 버그 수정·테스트·배포를 각각 다른 작업 단위로 분리하고, 실제 담당자가 바뀔 때는 HANDOFF에 현재 브랜치, 커밋, 검증, 미검증, 다음 위험만 남긴다. 여러 AI를 쓰는 목적은 답을 많이 받는 것이 아니라 서로 다른 성격의 일을 같은 품질 기준으로 연결하는 것이다.

모델과 추론 강도도 작업 조건으로 명시한다. 물리 상태 경쟁, 구조 리팩터링, 출시 판정처럼 실패 비용이 큰 일은 GPT-5.6 Sol / High 수준의 깊은 추론을 사용하고, 단순 문구나 기계적 수정은 더 가벼운 작업으로 분리한다. 모델을 숨은 변수로 두지 않아 작업자가 기대하는 정밀도와 비용을 맞춘다.

입력 포맷

스크린샷 + 영상 + 기대 동작 + 금지 조건

실행 단위

되돌릴 수 있는 작은 슬라이스 + 한 명의 주 수정 담당

완료 정의

플레이 확인 + 자동 테스트 + 실제 공개 번들

감각 언어를 버리지 않고, 판정 가능한 규칙으로 바꾼다

‘억까’, ‘어색함’, ‘웃겨야 함’은 모호해서 쓸 수 없는 말이 아니라 게임 디자인 의도가 들어 있는 입력이다. 개발 단계에서는 이를 재현 조건으로 한 번 더 번역한다.

플레이 중 표현	디자인 의도	구현 가능한 조건	검증
“이동이 커서 억까다”	내 실수보다 입력 해상도가 결과를 좌우하면 안 됨	기본 0.08m, Shift 미세 이동 0.02m	같은 카메라 방향에서 한 칸 입력량 비교
“맨바닥에 놓이면 롤이 이상하다”	한 탑을 이어 쌓아야 함	현재 탑의 수평 범위 안에서만 이동, 실제 탑 충돌 없으면 실패	기초 3개 밖 샘플과 겹침 경계 테스트
“내 블록 하나만 떨어졌다”	실패는 감점과 명확한 결과로 이어져야 함	동적 블록 AABB 바닥면이 경기장 높이에 닿으면 이탈	중심은 높지만 긴 블록 끝이 바닥에 닿는 회귀 테스트
“젠가 성공했는데 게임오버”	젠가 성공 판정과 일반 붕괴 감시가 경쟁하면 안 됨	pull/settling 동안 젠가 판정기가 결과 소유	수동 추출 뒤 높이 감소가 중복 철거를 부르지 않음
“문어가 꼭 잡았으면”	특수 블록의 역할이 한눈에 보여야 함	아래 지지 블록 탐색, LockConstraint, 다리 감김 애니메이션	측면 충격·대상 크기·표정 반응 테스트

이 번역에서 중요한 것은 수치를 먼저 정하는 것이 아니다. 먼저 **어떤 실수를 플레이어 책임으로 볼지**와 **어떤 결과를 시스템 책임으로 볼지**를 구분한다. 그 뒤 0.08m, 4.2m/s 같은 값은 의도를 만족시키는 가설로 넣고 플레이와 테스트로 조정한다. 따라서 수치는 언제든 바뀔 수 있지만, ‘정상적인 배치는 탑과 실제로 접촉해야 한다’는 규칙은 쉽게 흔들리지 않는다.

또한 부정 조건을 적극적으로 기록한다. 다음 블록 미리보기 금지, 원형 블록 금지, 삼각형 제거, 철거 결산 뒤 중복 감점 금지, 채팅 로그에 시스템 메시지 삽입 금지처럼 ‘하지 말아야 할 것’이 많았다. 시는 긍정 요구만 받으면 비슷해 보이는 기능을 과하게 추가하기 쉽기 때문에, 금지 조건은 게임의 정체성과 회귀 방지를 동시에 지키는 사양이다.

내가 유지한 판단 기준

긴장감은 남기되 조작 해상도와 숨은 보정 때문에 억울하지 않을 것. 실패는 웃길 수 있지만, 결과가 왜 발생했는지는 플레이어가 화면에서 이해할 수 있을 것.

물리 난이도는 값 하나가 아니라 책임 경계의 합이다

혼자 플레이했을 때 10-20m가 한계였던 장면을 기준으로, 50-100m까지는 실력과 판단을 시험하되 시스템 때문에 억울하지 않은 목표를 세웠다.

0.08m 기본 이동	0.02m Shift 미세 이동	4.2m/s 낙하 속도 상한	0.32s 첫 충돌 완충	5% 문어 등장 확률
-----------------------	-----------------------------	---------------------------	-------------------------	-----------------------

조정 축	게임적 허용	보정의 한계
입력	미세 이동을 별도 제공해 화면 원근 오차를 보완	자동 중앙 정렬로 플레이어 결정을 빼앗지 않음
낙하	낙하 속도 상한과 첫 충돌 충격 흡수	실제 탑 접촉 없이는 성공으로 인정하지 않음
안정 보조	약한 겹침에서 최대 0.08m만 지지면 중심 쪽으로 보정	0.34m 이상 늘어나거나 크게 기울면 제약 해제
젠가	선택 블록 마찰 0.08, 상단 바디를 깨워 공중 부양 방지	최상단·기초층 제외, 충분한 높이 없으면 자동 스킵
블록 분포	쉬운 직육면체를 자주, 어려운 복셀 블록을 희귀하게	특수 문어도 5%로 제한해 보조가 전략을 대체하지 않음

처음에는 난이도를 낮추기 위해 안정 제약을 강하게 걸었지만, 가느다란 면으로 선 블록이 물리적으로 무너지지 않는 장면이 생겼다. 이 결과는 '쉬워짐'이 아니라 물리 규칙에 대한 신뢰를 깨는 것이므로 롤백 대상이었다. 이후 보조는 **약한 겹침을 살리는 짧은 구간**에만 작동하고, 큰 기울기와 지지 이탈에서는 즉시 실제 물리에 양보하도록 경계를 정했다.

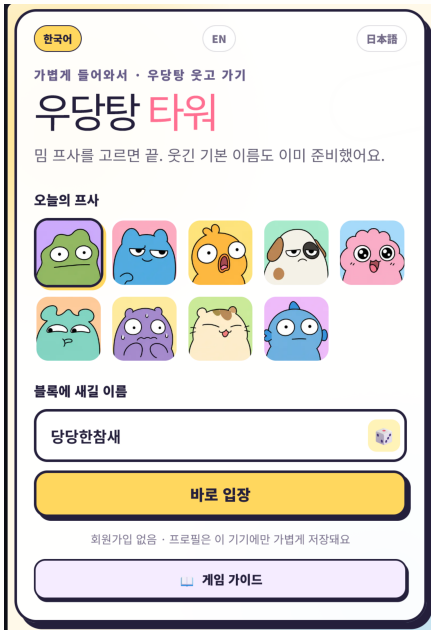
50m 연속 배치 테스트는 물리가 무조건 안정해야 한다는 뜻이 아니다. 부드러운 중앙 배치가 시스템 오차로 무너지는지 확인하는 기준이며, 강한 측면 충격과 영성한 가로 배치는 여전히 붕괴할 수 있어야 한다. 성공 가능성과 붕괴 가능성을 동시에 유지하는 것이 이 게임의 물리 품질이다.

플레이 테스트 원칙

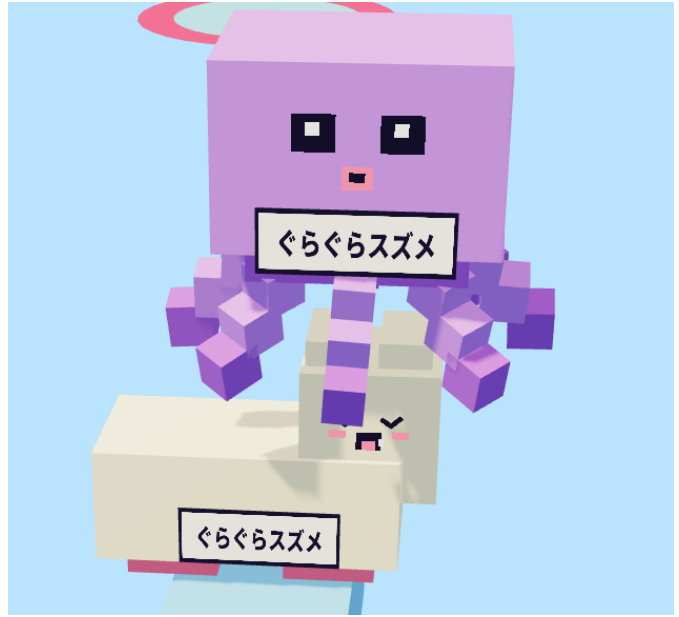
보정은 실수를 지우는 치트가 아니라, 입력과 화면 표현의 오차를 줄이는 안전망이다. 플레이어가 만든 불안정성은 끝까지 물리에 남겨 둔다.

이미지를 '생성'시키는 것이 아니라 게임 규칙에 맞게 제작시킨다

AI 리소스는 한 번에 채택하지 않았다. 참고 이미지에서 감정과 구조를 분리하고, 화면 크기·성능·충돌 형태까지 조건으로 넣어 반복했다.



뽀빠이 프로필과 자동 닉네임으로 진입 선택 피로를 줄인 입장 화면.



문어 다리가 실제 지지 블록의 크기에 맞춰 감춰 감기고, 아래 블록 표정이 반응하는 특수 블록 연출.

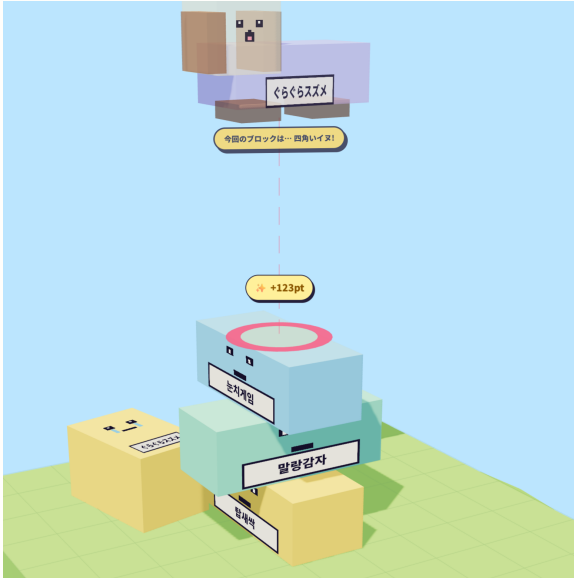
단계	내가 지시한 기준	반복 과정에서 생긴 기술 판단
레퍼런스 해석	Pepe류의 뽀빠이 감정, 귀엽고 즉시 읽히는 실루엣	기존 캐릭터 복제가 아니라 감정 범주와 형태 언어만 추출
형태 제약	원형·삼각형 금지, 마인크래프트처럼 박스로 읽히는 복셀	충돌체도 Box 조합만 사용해 보이는 형태와 물리 형태를 맞춤
오류 피드백	돌출된 눈·입 금지, 얼굴은 표면 내부 색면, 침대·머리 Z-fighting 제거	스크린샷에서 잘못된 면과 깊이를 표시해 수정 조건으로 전달
게임 연결	문어는 아래 블록을 꼭 잡고, 고양이·개는 마사지 받는 표정	지원 블록 탐색·제약·다리 애니메이션·표정 상태를 하나의 규칙으로 통합
운영 제약	모바일 로딩 최소화, 큰 캐릭터 애니메이션 불필요	프사는 스프라이트 시트, 3D 리소스는 코드 기반 복셀과 단순 재질 사용

이 방식에서 사용자는 AI가 내놓은 여러 그림 중 마음에 드는 것을 고르는 아트 소비자가 아니다. **게임 안에서 리소스가 해야 할 일**을 정의하고, 시각 결과가 그 기능을 수행하지 못할 때 이유를 설명하는 디렉터다. 캐릭터 표정은 장식이 아니라 진입 장벽과 소셜 분위기를 전달하고, 블록 실루엣은 재미뿐 아니라 쌓기 난이도와 충돌 신뢰도를 결정한다.

특히 문어는 '웃긴 블록 하나 추가'에서 끝나지 않았다. 5% 확률, 아래 지지 블록 선택, 물리적 일체화, 젠가에서 함께 빠지는 규칙, 감긴 다리의 시각 피드백, 대상 종류별 표정 반응까지 연결되면서 콘텐츠·물리·연출·테스트가 같은 사양을 공유하게 됐다.

스크린샷 한 장을 재현 가능한 버그 명세로 바꾼다

화면에 보이는 사실, 코드가 잘못 판단한 이유, 정상 기대값을 분리하면 시도 '비슷해 보이는 수정'이 아니라 정확한 회귀 방지를 수행할 수 있다.



CASE A · 바닥에 닿았는데 화살이 오지 않음

관찰 · 노란 블록이 경기장 바닥에 완전히 내려왔지만 점수 감점과 화살 철거가 시작되지 않았다. 블록 중심은 여전히 높고 X/Z 경기장 범위 안이어서 기존 판정이 이탈을 놓쳤다.

규칙 재정의 · 동적 블록은 중심점이 아니라 충돌체의 AABB 바닥면이 플랫폼 높이 0.08m 이하에 닿으면 탑 이탈이다. 초기 정적 기초 블록은 예외다.

수정과 검증 · 공용 isBlockLostFromTower 규칙을 고치고, 중심은 높지만 긴 블록 끝이 바닥에 닿은 입력을 테스트로 추가했다. 이후 감점 -> 당황 표정 -> 화살 비행 -> 충격 -> 결과 카드 순서를 별도 모듈과 테스트로 확인했다.

CASE B · 젠가에 성공했는데 일반 붕괴 판정이 게임오버를 선언

젠가 블록을 성공적으로 뺀 직후 탑 높이가 잠시 줄어드는 것은 정상이다. 하지만 일반 붕괴 감시가 같은 프레임의 높이 감소를 먼저 읽으면 성공 결과와 게임오버가 경쟁한다. 이를 단순 시간 지연으로 덮지 않고, **jenga pull/settling 동안에는 젠가 전용 판정기만 결과를 소유하도록** 상태 경계를 바꿨다. 일반 감시는 젠가 결산이 끝난 뒤 다시 시작한다.

이 회귀는 'passive collapse monitoring yields to the Jenga settlement judge' 테스트로 고정했다. 테스트 이름 자체가 기능보다 책임 관계를 설명하도록 해, 네트워크 버전에서 서버 판정으로 옮길 때도 의도를 보존한다.

이런 버그는 코드를 많이 읽는 것만으로는 빠르게 찾기 어렵다. 플레이어가 본 장면에는 이미 상태 경쟁의 결과가 드러나 있기 때문이다. 나는 캡처에 '이것만 떨어졌음', '젠가 성공 상태', '화살이 없음'처럼 사실을 붙이고, AI는 그 사실을 만족시키지 못하는 판정 함수를 역추적한다. 수정 뒤에는 같은 입력을 자동화된 규칙 테스트로 남겨 대화 맥락이 사라져도 기준이 유지되게 한다.

버그 보고 포맷

관찰된 화면 + 직전 행동 + 정상 기대값 + 예외 조건 + 재현 빈도 + 롤백 허용 여부

빠른 구현을 유지보수 가능한 경계로 되돌린다

AI가 변경 속도를 높일수록 규칙의 소유 위치와 회귀 검증을 더 명시적으로 관리했다. 다만 남은 부채도 숨기지 않는다.

입력·카메라

GameControlsPanel · ProfileGate
· 모바일/PC 입력

규칙·물리

gameplayRules
placementPhysics
jengaPhysics · blockPhysics

콘텐츠·연출

blockCatalog · blockVisuals ·
voxelDecorations · octopusSpecial

TowerGame.tsx

월드 루프와 상태 전환을 조율하는
현재 오케스트레이터

UI·언어

GameGuide · i18n · globals.css ·
KO/EN/JP

결과 연출

demolitionArrow · 점수 · 해설 · 오
디오

검증·출시

tests · build · Cloudflare ·
Git/HANDOFF

처음에는 한 TSX에 물리, 입력, UI, 오디오, 블록 정의가 함께 자라면서 수정의 영향 범위가 커졌다. 네트워크를 넣기 전에 규칙을 순수 함수와 작은 물리 모듈로 분리했고, 특수 문어·젠가·배치 보조·화살 연출도 독립 파일과 테스트를 갖게 했다. 이 구조 덕분에 ‘바닥 이탈 판정’은 렌더링 코드가 아니라 gameplayRules에서 수정하고, ‘화살이 실제로 날아오는 시간’은 demolitionArrow 단위 테스트로 검증할 수 있다.

하지만 **TowerGame.tsx**는 **현재도 약 2,562줄**로 크다. 물리 루프의 연결점과 UI 상태가 아직 한 컴포넌트에 남아 있어, 이 문서에서는 모듈화가 끝났다고 과장하지 않는다. 다음 단계는 월드 세션, 턴 상태 머신, 점수/결산, 카메라 컨트롤러를 분리하고, 서버 권한 이벤트와 클라이언트 표현을 다른 계층으로 나누는 것이다.

검증 계층	현재 기준	막는 회귀
규칙·물리 단위 테스트	31개	배치 범위, 점수, 젠가, 문어, 화살, 낙하 보조
렌더링·제품 요구 테스트	2개	경량 입장, 중량 게임 지연 로드, UI/언어 요구
정적 검사	TypeScript noEmit · ESLint · diff check	타입 불일치, 미사용 경로, 잘못된 패치
프로덕션 확인	vinext build + 공개 Worker 번들	개발 서버만 통과하고 공개판이 낡는 문제

남은 기술 부채를 문서에 쓰는 이유

AI 어시스트의 신뢰는 ‘완성됐다’는 말보다 현재 경계, 검증 범위, 미검증 위험을 정확히 공개할 때 높아진다.

대화의 끝이 아니라 공개 빌드 확인까지가 한 작업이다

코드가 수정됐다는 답변만으로 완료 처리하지 않는다. 저장소 상태, 자동 검증, 커밋, 원격 푸시, 실제 Cloudflare 공개판을 순서대로 확인한다.

Gate	확인 항목	실패하면
1. Scope	사용자 변경과 작업 중 변경을 보존하고 이번 슬라이스만 diff에 포함	범위 밖 변경을 분리하거나 중단
2. Static	TypeScript, ESLint, git diff --check	현재 슬라이스 안에서 수정
3. Behavior	규칙·물리 테스트 31개, 제품 요구 테스트 2개	새 기능으로 넘어가지 않고 회귀 원인 해결
4. Production	vinext 프로덕션 빌드	개발 서버와 번들 차이 진단
5. Version	기능 단위 커밋, 현재 브랜치와 HEAD 기록	모호한 대형 커밋을 쪼갬
6. Publish	GitHub push, Cloudflare deploy, 공개 URL 확인	원격 번들·캐시·배포 버전 점검
7. Handoff	변경, 검증, 미검증, 위험, 다음 작업만 갱신	대화 기록 대신 현재 상태를 다시 정리

프로토타입 단계에서는 빠르게 실험했지만, 난이도와 물리 규칙이 기준점에 도달한 뒤 버전을 **0.1.0**으로 고정했다. 이후 커밋은 특수 블록, 난이도, 모바일 UI, 젠가, 화살 연출처럼 독립적인 목적을 갖는다. 잘못된 카메라 추적이나 과도한 안정 보조는 새 구현 위에 보정하지 않고 롤백해, 결정의 역사를 Git에서 추적할 수 있게 했다.

AI가 배포 명령을 실행할 수 있어도 출시 판단은 자동화하지 않는다. 사용자가 '푸시·배포'를 요청한 범위에서만 외부 상태를 바꾸고, 공개 URL에서 결과를 다시 확인한다. 인증 정보와 빌드 캐시는 저장소에서 제외하며, 배포 버전이 소스 HEAD와 다르면 완료로 보고하지 않는다.

966676d

현재 게임 HEAD

0.1.0

패키지 버전

33

자동 검증 항목

완료의 정의

요구사항이 코드에 반영됨 + 회귀 테스트 통과 + 프로덕션 빌드 성공 + Git 이력 보존 + 실제 공개판에서 확인 가능

싱글의 물리 기준을 세 가지 플레이 모드로 확장한다

현재 로컬 플레이에서 검증한 규칙을 그대로 네트워크에 복사하지 않는다. 서버 권한, 지연, 재접속, 관전자를 고려해 상태의 주인을 다시 설계한다.

01 · 싱글 모드

현재 기준 환경. 물리·조작·점수 루프를 혼자 반복하며 서버 없이도 게임 감각을 검증한다.

02 · 프라이빗 룸

친구 초대 링크로 입장. 차례, 채팅, 젠가 이벤트, 팀 기록을 소규모 방에서 공유한다.

03 · 공개 멀티

최대 20인이 한 탑을 함께 쌓고, 실시간 점수·관전·채팅으로 공개 방의 소셜 재미를 만든다.

설계 과제	현재 로컬 기준	네트워크 버전에서 바뀌는 점
턴 권한	클라이언트 타이머와 입력	서버가 순서·60초·스킵·젠가 담당자를 확정
물리 결과	한 브라우저의 cannon-es 월드	서버 판정 또는 검증 가능한 이벤트로 배치·이탈·붕괴 확정
상태 동기화	로컬 BlockRecord	블록 ID, 변환, 소유자 스냅샷, 점수 이벤트를 대역폭에 맞춰 전송
계정	로컬 저장 게스트	게스트 토큰, 중복 닉네임, 재접속, 이후 영구 계정 연동
사회적 기능	로컬 채팅·랭킹·해설	20인 채팅, 관전, 리액션 스팸 제한, 팀 명예의 전당
복구	페이지 새로고침 시 로컬 복원	방 스냅샷, 중도 입장, 연결 끊김과 호스트 부재 처리

가장 중요한 전환은 물리를 서버에서 그대로 재현하는 것이 아니라 **어떤 결과를 서버가 권위 있게 확정해야 하는지**를 정하는 일이다. 예를 들어 클라이언트는 블록을 조작하고 시각적으로 예측할 수 있지만, 성공 배치, 경기장 이탈, 점수, 젠가 성공, 구조 붕괴는 서버가 동일한 순서로 확정해야 한다. 그래야 화살 연출과 결과 카드도 모든 플레이어에게 같은 사건으로 보인다.

v0.1에서 규칙과 연출을 분리한 이유가 여기서 드러난다. gameplayRules의 판정은 서버 공용 규칙으로 옮길 수 있고, demolitionArrow와 표정 변화는 확정 이벤트를 소비하는 클라이언트 표현으로 남길 수 있다. 다음 버전은 기능 추가보다 이 권한 경계를 먼저 검증한 뒤 프라이빗 룸, 공개 멀티 순으로 확장할 계획이다.

다음 버전의 첫 검증 질문

지연과 재접속이 있어도 모든 사용자가 '같은 블록이 같은 이유로 성공하거나 무너졌다'고 이해할 수 있는가?

내가 AI를 쓰는 방식은 '빠른 생성'보다 '빠른 검증'에 가깝다

게임 프로그래머로서 재미와 기술을 분리하지 않고, 감각을 사양으로 만들고 사양을 다시 플레이로 확인하는 순환을 운영한다.

원칙	실제 행동	포트폴리오에서 보이는 증거
판단을 먼저 둔다	기능보다 재미의 이유, 허용/금지 조건, 실패 책임을 먼저 말한다	배치 범위, 붕괴 분류, 젠가 결과 소유
감각을 보존한다	'억까', '어색함', '웃김'을 버리지 않고 측정 가능한 조건으로 번역한다	미세 이동, 충격 완충, 복셀 규칙, 표정 반응
AI를 역할로 나눈다	리소스, 통합, 범위 수정, 테스트·배포를 독립 작업 단위로 관리한다	작은 커밋, 모듈, HANDOFF, 공개 URL
증거로 대화한다	스크린샷과 영상에 관찰·기대·예외를 붙인다	바닥 이탈, 젠가 경쟁, 모바일 UI 회귀
틀리면 롤백한다	결과가 비슷해도 체감이 나쁘거나 물리를 속이면 되돌린다	카메라 추적 롤백, 과한 안정 보조 제거
완료를 공개판으로 정의한다	테스트와 빌드 뒤 GitHub-Cloudflare 실제 결과를 확인한다	v0.1, 33개 자동 검증, 배포 이력

이 프로젝트는 AI 어시스트로 빠르게 만들어졌다. 그 사실을 숨기지 않는다. 대신 결과물의 품질을 설명할 때 'AI가 잘 만들었다'는 말에 기대지 않는다. 어떤 장면에서 문제가 드러났고, 내가 어떤 규칙을 정했으며, 구현을 어떻게 좁혀 검증했고, 무엇을 아직 기술 부채로 남겼는지를 공개한다. 이것이 내가 생각하는 개발자 지향 바이브코딩이다.

AI는 반복 비용을 낮춰 더 많은 플레이 테스트와 더 빠른 실험을 가능하게 한다. 게임 프로그래머는 그 여유를 더 많은 기능에만 쓰는 것이 아니라, 재미의 책임 경계를 분명히 하고 실패를 재현하며 다음 버전이 감당할 구조를 만드는 데 써야 한다. 우당탕 타워 v0.1은 그 작업 방식의 첫 공개 결과다.

PORTFOLIO TAKEAWAY

AI에게 정답을 받은 프로젝트가 아니라, AI가 빠르게 움직여도 게임의 방향과 품질이 흔들리지 않도록 개발 루프를 설계한 프로젝트.

GitHub · Play Build · Gameplay Short

우당탕 타워 v0.1 · 박찬욱 · Game Client Programmer · 2026.08.02